R → Python

Python for R users

A hands-on workshop for people who already analyze data

Ethan Meyers

1:00 – 5:00 PM

Plan for today

SESSION 1 — 1:00 – 3:00

Welcome, introductions, environment setup

Python basics: syntax and data structures

NumPy: array computations

Matplotlib: core plotting

BREAK AT 3:00

SESSION 2 — 3:15 – 5:00

Pandas and Seaborn

Statistical modeling: statsmodels, scipy.stats

Machine learning: scikit-learn

Object-oriented programming and duck typing

Format: short slide segments, then live coding in Jupyter

Workshop goals

GOAL 1

Do the analyses you already do in R — in Python

The Python equivalents of your R workflow

Pitfalls that catch R users: indexing, copies, etc.

GOAL 2

See what Python adds

The machine learning ecosystem and object-oriented programming

Aim: transition your own analyses (and perhaps teach a data science class) in Python

Assumed: Experience with base R plus the main tidyverse packages (dplyr, ggplot2)

Python vs. R

R IS STRONG AT

Built by statisticians: plotting, vectorized computation, and data frames, etc., inherent part of the language

Strong ecosystem for statistical methods (CRAN) and reporting (Quarto)

Easier to get started analyzing data quickly — an advantage for teaching

PYTHON IS STRONG AT

Easier to build larger packages and integrate into software, pipelines, and production

A substantially more mature machine learning ecosystem, the default language of AI tooling

Larger user base

Useful to know both: each language shows you what the other adds or is missing

Today's approach

Native

WE TAKE THIS PATH

The tools Python users most often use

`Jupyter` · `pandas` · `matplotlib` · `seaborn`

Ex-pat

NOT TODAY

R idioms recreated in Python

`Quarto`, `polars` (dplyr-like) · `plotnine` (ggplot2 clone)

Rationale: You will need to read commonly used code, and the ex-pat packages are easier to learn on your own

Introductions

YOUR TURN · 2 MINUTES

Introduce yourself to your neighbors

Name, what you work on, and what you want out of Python

00

Python environments

COVERS Per-project environments · uv · getting JupyterLab running

Installing packages

IN R

`install.packages()` puts every package into one system-wide library, and that mostly works fine

Alternative renv (and pak) where each project specifies which package version to use

IN PYTHON

Your operating system itself runs Python so changing the system copy can break things

We will use an approach where each project gets its own **environment** of packages

conda

Named environments that live system-wide and can hold non-Python software too (even R)

Can be very (very) slow

.venv + uv

WHAT WE USE

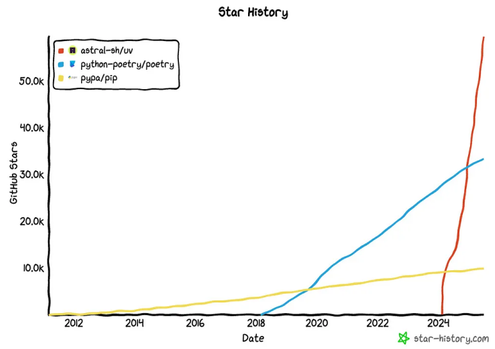
A `.venv` folder inside each project. Traditionally managed with pip + `requirements.txt`

uv is the modern tool: much faster, and it manages versions and conflicts. Think `renv` + `pak`

⚠ **PITFALL** “Python can’t find my package” usually means didn't active the correct environment

Using uv

`uv init`	Start a project: creates `pyproject.toml`, `uv.lock` and a git repo
`uv add pandas`	Install a package into `.venv`, recording it in `pyproject.toml` + `uv.lock`
`uv sync`	Install everything a project lists; e.g. after downloading someone else's code
`uv run script.py`	Run a script using the project's environment



GITHUB STARS — UV HAS OVERTAKEN PIP AND POETRY

Two main files:

- `pyproject.toml`: what you asked for (e.g. `"requests>=2.0"`)
- `uv.lock` = the exact resolved version of every package, managed by uv

PYPROJECT.TOML

```
[project]
name = "workshop_notebooks"
version = "0.1.0"
requires-python = ">=3.13.2"
dependencies = [
"numpy",
"scipy",
"pandas",
"matplotlib",
"seaborn",
"statsmodels",
"scikit-learn",
"jupyterlab",
]
```

Where do you write Python?

In R nearly everyone uses RStudio or Positron. Python has many more choices:

TOOL	STRENGTHS	TYPICAL USERS
VS Code	Most popular — flexible, extensions for everything	Developers, data scientists
Positron	RStudio's successor — one IDE for R and Python, Quarto built in	People coming from RStudio
PyCharm	Deep Python tooling and debugging	Python developers
Spyder	MATLAB-like layout, variable explorer, IPython console	Scientists, engineers
JupyterLab TODAY	Interactive notebooks in the browser	Data scientists, researchers, students
Google Colab	Jupyter in the cloud — zero setup, free GPUs	Students, quick starts, sharing

JUPYTER VS QUARTO **`Jupyter .ipynb`** are JSON with outputs saved inside; **`Quarto .qmd`** is plain markdown, rendered fresh

Jupyter notebook cells run in any order, so state can drift. Restart & Run All before trusting results

- *Marimo notebooks* are a newer reactive notebook that keeps state consistent between code cells

Getting started

1. Before today you ran the setup script: it installed packages (`uv sync`) and allows you to launch JupyterLab

2. Fallback Google Colab: Allows you to run Jupyter notebooks hosted on the Internet

3. Open the first notebook: **`notebook_01.ipynb`**

⚠ **PITFALL** Only the last value in a code cell is displayed. Use `print()` to print additional values in a cell

01

Python basics

COVERS Syntax · strings · imports · lists · dictionaries · loops · functions

Syntax and types

	R	PYTHON
Assignment	`a <- 7`	`a = 7`
Exponentiation	`2^3`	`2**3`
Booleans	`TRUE, FALSE`	`True, False`
Null value	`NULL`	`None`
Missing values	`NA`	none built in — NumPy/Pandas use `NaN`
Checking types	`class(a)`	`type(a)`

⚠ **PITFALL** `^` is bitwise XOR in Python: `2^3` evaluates to 1, not 8. Always use `**`.

▶ LET'S TRY IT IN PYTHON

Strings and methods

Strings are created the same way in both languages: `s = "my text"` or `s = 'my text'`

	R	PYTHON	
Insert a value into a string	`paste("n is", val)`	`f"n is {val}"`	similar to R's glue::glue()
String length	`nchar(s)`	`len(s)`	
Change case	`toupper(s)`	`s.upper()`	
Replace text	`gsub("a", "b", s)`	`s.replace("a", "b")`	
Split on a delimiter	`strsplit(s, ",")`	`s.split(",")`	

Python attaches functions to objects: these are called **methods**

To discover them: type `s.` then Tab

▶ LET'S TRY IT IN PYTHON

Import statements

Where `library()` loads extra functions in R, `import` adds functionality from a **module** (roughly, a `.py` file).

A **package** in Python is a collection of modules, just like a package in R is a collection of .R files

LOADING	USE
`import math`	`math.sqrt(81)`
`import math as m`	`m.sqrt(81)`
`from math import sqrt`	`sqrt(81)`
`from math import *`	like `library()` — strongly discouraged!

THE STANDARD ALIASES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

To repeat: Namespaces usually stay attached

- As if you always wrote `pkg::fun()` in R

▶ LET'S TRY IT IN PYTHON

Lists

The closest analog to an R vector, but...

- 0-based indexing
- Not vectorized
- Can hold mixed types

	R VECTOR		PYTHON LIST	
First element	`v[1]`		`my_list[0]`	
Last element	`v[length(v)]`		`my_list[-1]`	
Slice (2nd–4th)	`v[2:4]`	Includes 4	`my_list[1:4]`	Excludes 4!
Vectorized math	`v + 1`	Works	`my_list + 1`	TypeError
Mixed types	coerced to one type		`[2, "a", True]`	allowed as-is

List pitfalls!

⚠ ASSIGNMENT IS NOT A COPY

```
a = [1, 2, 3]
b = a
b[2] = 99      # a[2] is now 99 too!

b = a.copy()  # the fix: ask for a copy
```

⚠ IN-PLACE METHODS RETURN NONE

```
v2 = v.sort()    # sorts v;  v2 is None!

# the fix: a sorted copy
v_sorted = sorted(v)
```

R’s copy-on-modify protects you from both. Python makes you *ask* for copies.

This theme returns in NumPy (views) and Pandas (`.loc` assignment).

▶ LET'S TRY IT IN PYTHON

Tuples and dictionaries

TUPLES

Like lists, but immutable; i.e., values can't change

```
point = (3, 5)

point[0]

# unpacking
a, b = point

lo, hi = min_max(data)
```

DICTIONARIES

Key → value lookup, like R's named vectors and lists

```
d = {"a": 1, "b": 2}

d["a"]   # look up a value

d["c"] = 3   # add a new entry

"a" in d # check for a key
```

▶ LET'S TRY IT IN PYTHON

For loops

R

```r
num_vec <- c()

for (i in 0:9) {
    num_vec[i + 1] <- i^2
}
```

PYTHON

```python
num_list = []
for i in range(10):
    num_list.append(i**2)


# the shortcut: list comprehension
[i**2 for i in range(10)]
```

`range(10)` runs from 0 to 9: the stop value is excluded

No braces: consistent indentation *is* the syntax

▶ LET'S TRY IT IN PYTHON

Functions

To create a function:

- `def` statement
- A colon
- Indentation for function body

Default argument values work like R's

`return` is explicit: no implicit last value like R

PYTHON

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

# multiple values return as a tuple
def sqr_cube(x):
    return (x**2, x**3)

x2, x3 = sqr_cube(3)
```

⚠ **PITFALL**

Never use a list or dict as a default value!

- Defaults are created once, at definition, and persist across calls
- Use `None` and create the object inside the function

▶ LET'S TRY IT IN PYTHON

02

Array computations with NumPy

COVERS Arrays · boolean indexing · missing values · random numbers

Arrays behave like R vectors

Lists are not vectorized: `np.array` is the solution!

```
import numpy as np
```

	R VECTOR	NUMPY NDARRAY
Create an array	`c(1, 2, 3)`	`np.array([1, 2, 3])`
Sequence with a step	`seq(0, 10, by=2)`	`np.arange(0, 11, 2)` stop excluded
Mean	`mean(v)`	`arr.mean()` or `np.mean(arr)`
Mean, NAs removed	`mean(v, na.rm=TRUE)`	`np.nanmean(arr)`

Boolean indexing and slicing

	R VECTOR	NUMPY NDARRAY
Filter by a condition	`v[v > 3]`	`arr[arr > 3]`
Combine conditions	`v[v > 1 & v < 4]`	`arr[(arr > 1) & (arr < 4)]`
Vectorized conditional	`ifelse(v > 3, "hi", "lo")`	`np.where(arr > 3, "hi", "lo")`
Slicing	`v2 <- v[3:10]`	`a2 = arr[2:9]` (returns a view)

⚠ **PITFALL**

Slices are views: modifying `a2` modifies `arr`

Use `.copy()` if you don't want to change the original

```
a2[0] = 7      changes arr too
arr[0]         will now return 7
```

Random numbers and seeds

NUMPY

Numpy's seeded **Generator object** is similar to R's global `set.seed()`

	R	NUMPY
Set a seed	`set.seed(42)`	`rng = np.random.default_rng(42)`
Normal draws	`rnorm(5)`	`rng.normal(0, 1, 5)`
Uniform draws	`runif(5)`	`rng.uniform(0, 1, 5)`
Sample w/o replacement	`sample(x, 3)`	`rng.choice(x, 3, replace=False)`

`dnorm` / `pnorm` / `qnorm` → **`scipy.stats`** (after the break)

▶ LET'S TRY IT IN PYTHON

03

Visualization with Matplotlib

COVERS The Figure – Axes model · core plot types

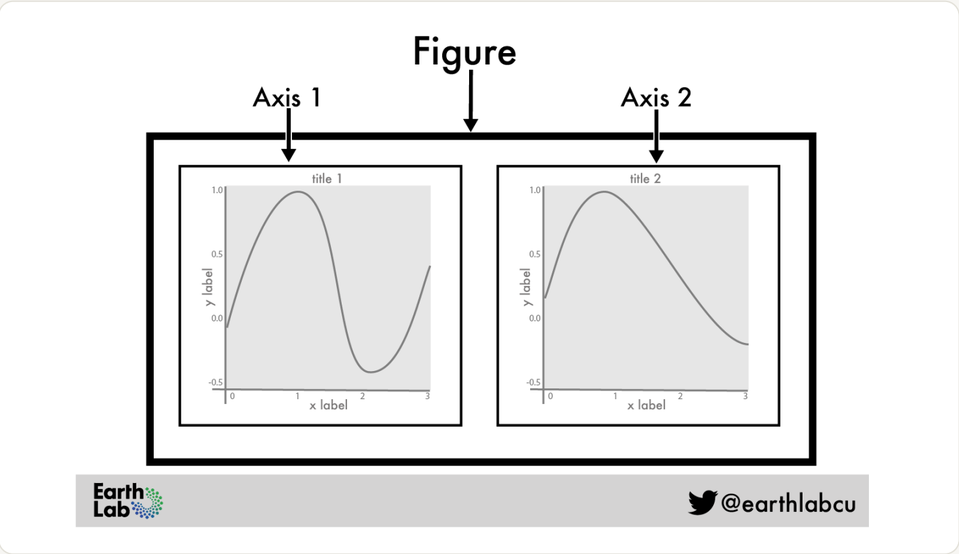
Figures and Axes

Matplotlib plots any sequence such lists, arrays, etc.

- This is analogous to Base R graphics
- Seaborn (after the break) operates on DataFrames

The **`subplots()`** function which returns:

- **Figure** (the canvas) holds one or more...
- **Axes** (the plots)



Plots are built element by element: create, add labels, add more data, ...

THE PATTERN

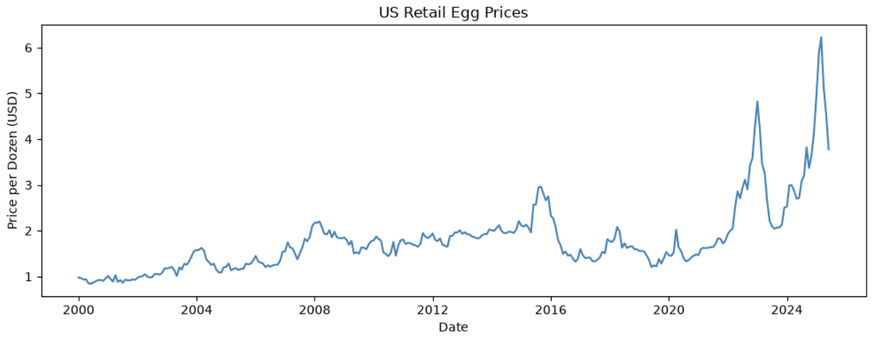
```
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_xlabel("Month")
ax.set_title("Egg prices")
plt.show()
```

Four core plot types

MATPLOTLIB

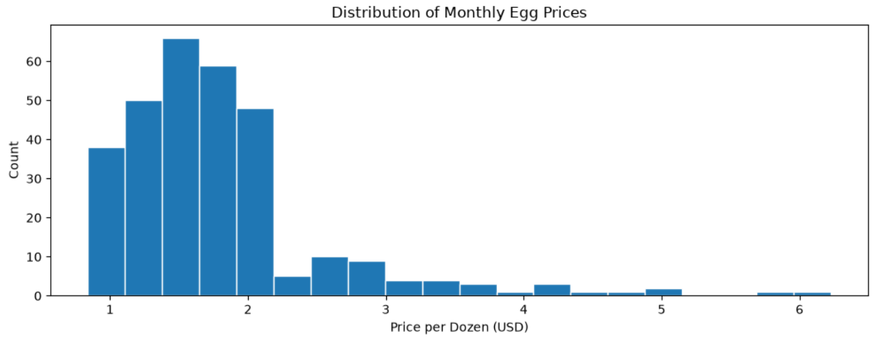
Line plot

`ax.plot(x, y)`



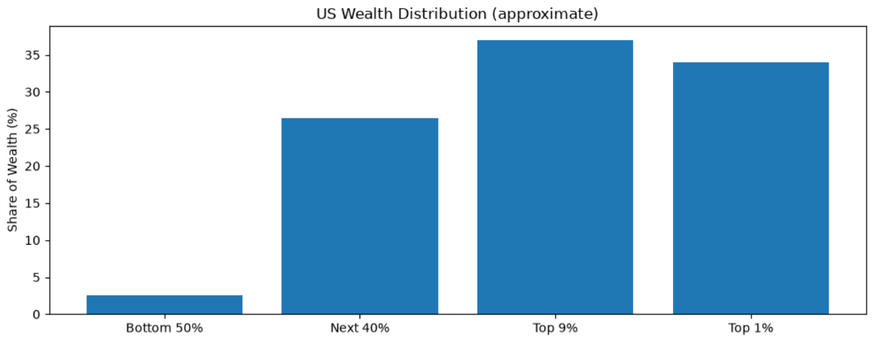
Histogram

`ax.hist(v)`



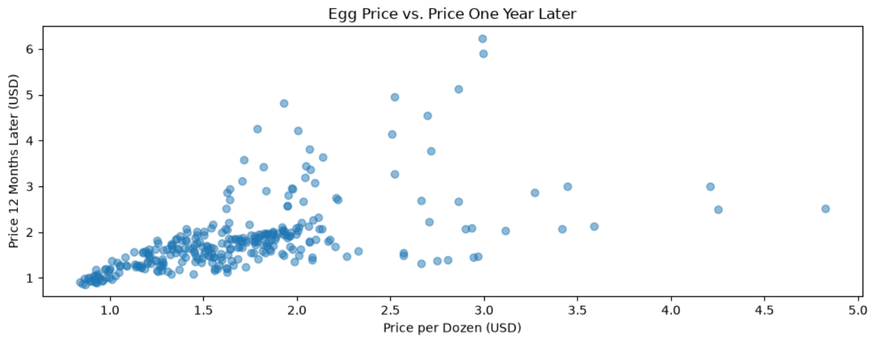
Bar plot

`ax.bar(x, y)`



Scatter plot

`ax.scatter(x, y)`



▶ LET'S TRY IT IN PYTHON

Break

BACK AT 3:15

UP NEXT Pandas · Seaborn · statistical modeling · scikit-learn · the object model

04

Data tables with Pandas

COVERS Series & the Index · selecting · the dplyr verbs · Seaborn

Pandas Series and the Index

Pandas is package for data analysis and manipulation

- Similar to dplyr

Pandas has two main data structures:

`Series` 1D labeled array ≈ a named R vector

`DataFrame` 2D table ≈ an R data frame

A Series is an ndarray plus an **Index** of row labels

Access elements:

`eggs.iloc[0]`	by **position**
`eggs.loc["2000-01-01"]`	by **Index label**

`ASIDE` *polars* is the modern alternative, but currently far less widely used than Pandas

EGGS — PRICE OF A DOZEN EGGS SERIES

date	price
2000-01-01	0.975
2000-02-01	0.962
2000-03-01	0.931
⋮	⋮

↑ INDEX ↑ ARRAY VALUES

▶ LET'S TRY IT IN PYTHON

DataFrames: selecting and filtering

Pandas DataFrames contain 2D data tables

Select one column → **Series**	`df["price"]`	
Several columns → **DataFrame**	`df[["date", "price"]]`	pass a list
Filter rows by values	`df[df["price"] > 3]`	or `df.query("price > 3")`
Combine conditions	`& |`	with parentheses - same rules as NumPy
Rows by position	`df.iloc[0:10]`	
Rows by Index label	`df.loc["2010-01-01":"2020-01-01"]`	

⚠ **PITFALL** `.iloc` slices exclude the stop; `.loc` slices include it.

The dplyr verbs, translated

PANDAS

	R · DPLYR	PANDAS
Add a column	`mutate(df, new = expr)`	`df["new"] = expr`
Grouped summary	`group_by(col) |> summarize()`	`df.groupby("year")["price"].mean()`
Sort	`arrange(df, desc(col))`	`df.sort_values("col", ascending=False)`
Pipelines	`df |> filter() |> summarize()`	`df.query(...).groupby(...).mean()`
Join on a key column	`left_join(df1, df2, by="key")`	`df1.merge(df2, on="key", how="left")`
Join on the Index	—	`df1.join(df2, how="left")`
Missing data	`na.omit(df)`	`df.dropna(), df.fillna(0)`

Aggregations skip NaN by default; i.e., `na.rm = TRUE` automatically used

▶ LET'S TRY IT IN PYTHON

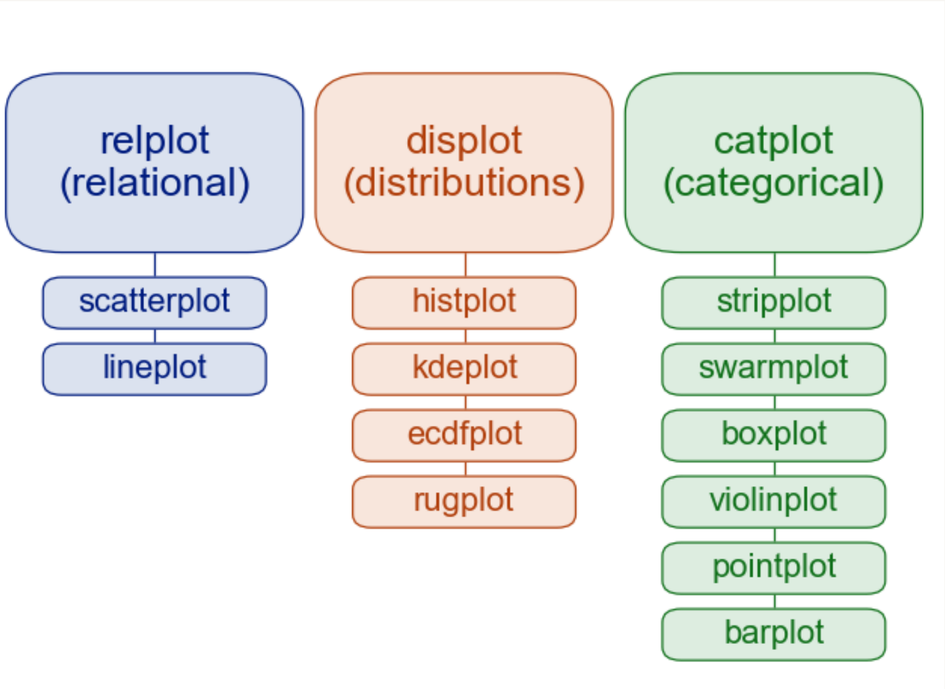
Statistical graphics with Seaborn

Seaborn provides a high-level interface for creating attractive and informative statistical graphics

- Built on Matplotlib, so everything from Part 4 still works

Figure-level functions that work directly on DataFrames, similar to ggplot2

`sns.relplot`	two quantitative variables
`sns.displot`	one quantitative variable
`sns.catplot`	quantitative across categories



THE ARGUMENTS: DATA=, X=, Y=, HUE=, COL=, KIND=

```
sns.catplot(data=econ, x="year", y="gas", kind="box")
```

▶ LET'S TRY IT IN PYTHON

05

Statistical modeling

COVERS statsmodels formulas · distributions in scipy.stats

StatsModels: R-style formulas

We can build linear models using the statsmodels package

- Very similar to R's `lm()` function


```
import statsmodels.formula.api as smf
```

	R	STATSMODELS
Fit a linear model	`lm(y ~ x, data=df)`	`smf.ols("y ~ x", data=df).fit()`
Model summary	`summary(model)`	`model.summary()`
Coefficients	`coef(model)`	`model.params`
Confidence intervals	`confint(model)`	`model.conf_int()`

ALSO `model.pvalues`, `model.rsquared`

Where dnorm, pnorm, qnorm live: scipy.stats

The scipy package provides a range of useful mathematical functions


```
from scipy import stats
```

	R	SCIPY.STATS
Density	`dnorm(x)`	`stats.norm.pdf(x)`
CDF	`pnorm(x)`	`stats.norm.cdf(x)`
Quantile	`qnorm(p)`	`stats.norm.ppf(p)`
Random draws	`rnorm(n)`	`stats.norm.rvs(size=n)`
Two-sample t-test	`t.test(x, y)`	`stats.ttest_ind(x, y)`

PATTERN The same four operations on every distribution: `stats.t`, `stats.chi2`, `stats.binom`, ...

▶ LET'S TRY IT IN PYTHON

06

Machine learning with scikit-learn

COVERS One API for every model · fit / predict / score

Scikit-learn: one API for every model

Scikit-learn is the industry standard for ML

- Connects to PyTorch, TensorFlow, Hugging Face

Consistent interface: switching algorithms means changing one line

No formulas, revolves around cross-validation:

- `X` is a 2D feature matrix (examples × features)
- `y` is a 1D array of responses

R's tidymodels follows a similar workflow, but sklearn has far wider coverage

▶ LET'S TRY IT IN PYTHON

THE WORKFLOW

```
# select a model
model = LinearRegression()

# train/test split
X_train, X_test, y_train, y_test =
    train_test_split(X, y)

# train → predict → evaluate
model.fit(X_train, y_train)
model.predict(X_test)
model.score(X_test, y_test)
```

07

The object model

COVERS Classes · dunder methods · duck typing

Object-oriented programming

Objected-oriented programming (OOP) is a paradigm built on **objects**, which contain:

`attributes` the data an object stores

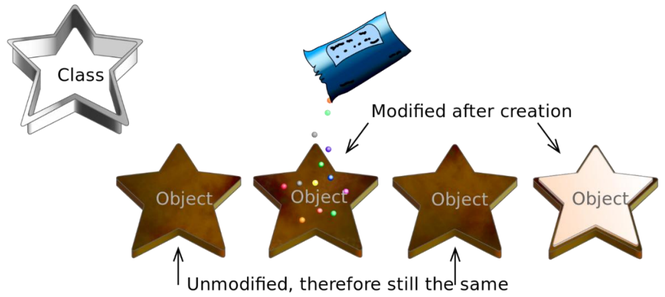
`methods` functions that operate on that data

Useful for stateful things:

- E.g., a fitted sklearn model stores its own state

`Class` defines the data and methods

`Object` an instance of a class, holding actual data



Python has **one coherent object model**

- Compared R's S3 / S4 / R6 / S7 patchwork

YOU'VE BEEN USING OBJECTS ALL DAY

```
# constructor (creates an object)
my_df = pd.DataFrame(
      {"a": [1, 2, 3],
      "b": [4, 5, 6]})

# method
my_df.sum()

# attribute
my_df.shape
```

Classes, methods, and constructors

A constructor defines what should happen when an object is created

- The constructor is defined with the `__init__(self)` method

CLASS CODE

```
class BaseballGame:

    # constructor
    def __init__(self):
        self.inning = 1
        self.end_of_game = False


    # method
    def print_inning(self):
        print(f"inning: {self.inning}")
```

CREATING OBJECT INSTANCES

```
# create a new object instance
my_game = BaseballGame()


# update an attribute
my_game.inning = 2


# call a method
my_game.print_inning()
```

Duck typing

“If it walks like a duck and quacks like a duck, it’s a duck”

Python checks whether the needed *methods exist,* not what class an object is

DUNDER METHOD	WIRES OBJECTS INTO THE SYNTAX
`__print__`	what is displayed when using the `print()` function
`__len__`	enables `len(obj)`
`__getitem__`	enables `obj[i]` — and for-loops

Implement `__len__` and `__getitem__` and your object automatically works with:

- `len()`
- Indexing using `[]`
- for-loops

No inheritance required!

▶ LET'S TRY IT IN PYTHON

Wrap-up

WE'VE COVERED

Python syntax and the pitfalls that catch R users

Core data structures: lists, tuples, dictionaries

NumPy, Matplotlib, Pandas, Seaborn

statsmodels & scipy.stats · scikit-learn

Classes, dunder methods, and duck typing

NEXT STEPS

Python for Data Science Handbook — free online

Python for Data Analysis — free online

The pandas and scikit-learn user guides

Use the R to Python cheat sheet as a useful reference

Practice!

Thanks — and happy Python-ing

Ethan Meyers